

27/06/2020, 14:38

Coursera

UNIVERSITY OF TORONTO

AI Agents and Agentic AI with Python & Generative AI

×

Module 1

Agentic AI Concepts

▼

Module 2

AI Agents, Tools, Actions, & Language

▼

Module 3

GAME, A Conceptual Framework for AI Agents

▼

Module 4

Agent Tool Management

▲

Keeping Agent Tools Up to Date with Python Decorators

Ungraded Plug-in • 12-min

Tool Organization for Agents

Ungraded Plug-in • 12-min

Refactoring Our REACT Agent

Ungraded Plug-in • 15-min

Try Out the REACT Agent with the Decorator Reading • 20-min

Module 5

Refactoring How Software is Built in the Age of AI Agents

▲

Build the Impossible with AI Agents

Video • 1 min

Refactoring How We Teach Innovation

Video • 4 min

The Inventory Management Agent

Ungraded Plug-in • 15-min

Introducing a New Form of Computing

Video • 7 min

New Ways to Access and Extract Information

Video • 3 min

The Inventory Management Agent

An Inventory Management Agent

Traditional software development involves writing complex code to handle every possible scenario. With LLM-powered agents, we can take a different approach: provide clear goals, simple tools, and let the agent's intelligence bridge the gap. Let's explore this through building an inventory management system.

Rethinking Software Architecture

Instead of building complex inventory logic, we can break down the system into simple tools and let the agent handle the complexity:

```
# Simple tools that focus on data operations
@register_tool(description="Save an item to inventory")
def save_item(action_context: ActionContext,
              item_name: str,
              description: str,
              condition: str,
              estimated_value: float) -> dict:
    """Save a single item to the inventory database."""
    inventory = action_context.get("inventory_db")
    item_id = str(uuid.uuid4())

    item = {
        "id": item_id,
        "name": item_name,
        "description": description,
        "condition": condition,
        "estimated_value": estimated_value,
        "added_date": datetime.now().isoformat()
    }

    inventory[item_id] = item
    return {"item_id": item_id}

@register_tool(description="Get all inventory items")
def get_inventory(action_context: ActionContext) -> list(dict):
    """Retrieve all items in the inventory."""
    inventory = action_context.get("inventory_db")
    return list(inventory.values())

@register_tool(description="Get specific inventory item")
def get_item(action_context: ActionContext, item_id: str) -> dict:
    """Retrieve a specific inventory item."""
    inventory = action_context.get("inventory_db")
    return inventory.get(item_id)
```

Notice how simple these tools are - they just handle basic CRUD operations. The intelligence comes from the agent's goals and system prompt:

```
# Define the agent's goals
goals = {
    Goal(
        name="inventory_management",
        description="""Maintain an accurate inventory of items including:
        - Detailed descriptions
        - Condition assessment
        - Value estimates
        - Historical tracking"""
    )
}

# Create the agent with clear instructions
agent = Agent(
    goals=goals,
    agent_language=JSONAgentLanguage(),
    action_registry=registry,
    capabilities=[
        SystemPromptCapability("""You are an expert inventory manager.
        When shown items:
        1. Identify the item type and key features
        2. Assess condition from visual cues
        3. Estimate market value based on condition and features
        4. Maintain organized records with consistent descriptions

        Always be thorough in descriptions and conservative in value estimates.""")
    ]
)
```

Using the System

Now users can interact with the system in natural language:

```
# Example interaction
result = agent.run("""I have a pair of Air Jordan basketball shoes.
They're red with the Jumpman logo, showing some wear
and slight discoloration.""")

# Agent might respond:
"""I'll help you add those shoes to inventory.

First, let me analyze the item details you've provided:
- Item: Air Jordan Basketball Shoes
- Color: Red
- Notable features: Jumpman logo
- Condition: Used with visible wear and discoloration

Based on these details and current market values, I'll create an inventory
entry.

Action: save_item
{
    "item_name": "Air Jordan Basketball Shoes",
    "description": "Red colorway with iconic Jumpman logo",
    "condition": "Used - visible wear and slight discoloration",
    "estimated_value": 85.00
}

The shoes have been added to inventory. Would you like to add any additional
items?"""
```

Extending with Images (Future Enhancement)

While not covered in our current implementation, this system could be extended to handle images with additional tools:

```
@register_tool(description="Analyze an image and describe what you see")
def process_image(action_context: ActionContext,
                 image_path: str) -> str:
    """
    Look at an image and describe the item, including type, condition, and
    notable features.
    Returns a natural language description.
    """
    with open(image_path, "rb") as image_file:
        image_data = base64.b64encode(image_file.read()).decode("utf-8")

    response = completion(
        model="gpt-4o",
        messages=[
            {
                "role": "user",
                "content": {
                    "type": "text",
                    "text": """Please describe this item for inventory
                    purposes.

                    Include details about:
                    - What the item is
                    - Its key features
                    - The condition it's in
                    - Any visible wear or damage
                    - Anything notable about it"""
                },
                {
                    "type": "image",
                    "image_url": {
                        "url": f"data:image/jpeg;base64,{image_data}"
                    }
                }
            ]
        ],
        max_tokens=1000
    )

    return response
```

Why This Approach Works

This architecture succeeds because:

- Simple Tools, Complex Understanding:** The tools handle basic operations while the LLM provides sophisticated analysis and decision-making.
- Natural Interaction:** Users can describe items in natural language or with photos instead of filling out structured forms.
- Flexible Intelligence:** The agent can:
 - Identify items from descriptions
 - Assess condition based on details provided
 - Estimate values using market knowledge
 - Maintain consistent record formats
- Easy Extension:** New capabilities can be added by:
 - Updating the system prompt
 - Adding simple tools
 - Enhancing the agent's goals

Real-World Applications

This pattern extends beyond inventory management. Consider:

- Policy compliance checking (like the travel expenses example)
- Document processing systems
- Customer service applications
- Data analysis tools

The key is always the same:

- Define clear goals and instructions
- Provide simple, focused tools
- Let the agent's intelligence handle complexity and allow adaptation

The future of software development isn't about writing every possible edge case - it's about providing the right framework for AI agents to handle complexity while keeping the tools and infrastructure simple and reliable.

↩ Expand

✓ Completed

Go to next lesson

Like Dislike Report an issue